

Exploring the Ability of a Linear Regression Model in Predicting Housing Prices

Mathematics: Analysis and Approaches HL
Internal Assessment

Nikita Manushin

June 2026

1 Introduction

1.1 Personal Connection

When I started learning math, in the very first grade, I have always wondered how connected, the numbers and the real world are. In the past few years Neural Networks became so powerful they could communicate like humans, all of that using tons of numbers, that create words and sentences. This linear regression model I am about to present, is an example, how what seem to be meaningless numbers can make predictions about something that humans care about.

1.2 Aims of the Investigation

In this paper, I will explore how a linear regression model can be trained using gradient descent and adjusted so that it predicts current prices of apartments in Amsterdam. I will look into three features of an apartment: living area, construction year and the distance from the center of Amsterdam. I will implement those features as independent variables into the model one by one, to see the influence of each of them on the price. The reason I am using gradient descent rather than ordinary least squares for solving my model is that I want to resemble a neural network, which is what I take my inspiration from for this model. Furthermore, this allows me to observe and explore how weights and biases are optimized iteratively using the gradients of the loss function.

1.3 Research Question

To what extent does increasing the number of input features for a linear regression model improve its accuracy at predicting housing current prices of apartments in Amsterdam.

2 What is a Linear Regression Model

2.1 A Simple Linear Regression Model

Neural Network is a human-brain inspired computing system that stacks simple “neurons” to achieve complex tasks. I see it as a method of approximating a complex process with a mathematical function.

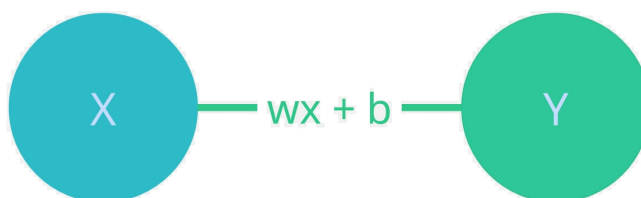


Figure 1: A Simple Neural Network with Two Neurons

Here is a simple Neural Network consisting of independent variable x and dependent variable y . The circles represent neurons, where neuron is just a variable and independent variable is called the input neuron and the dependent variable is called the output neuron. The line connecting them means that the function of y is $wX + b$, where w and b are constants; they are called weight and bias, respectively.

2.2 Machine Learning and Gradient Descent

The goal of machine learning is to find the specific weight and bias that will approximate a process with the least loss. Loss function is used to calculate how incorrect the network is. $L(y) = \frac{1}{2}(y - t)^2$, where y is the output of the network and t is the actual target answer. $(y - t)$ calculates the error the network has made, squaring removes any negative results and $\frac{1}{2}$ makes the derivative cleaner: $dL/dy = y - t$.

Machine learning is the process of finding the correct weights and biases that minimize loss. Let's take a look at this diagram of a neural network that learns to predict housing prices in Amsterdam from the area of the apartment. The input is the area, and the output is the price of the house.

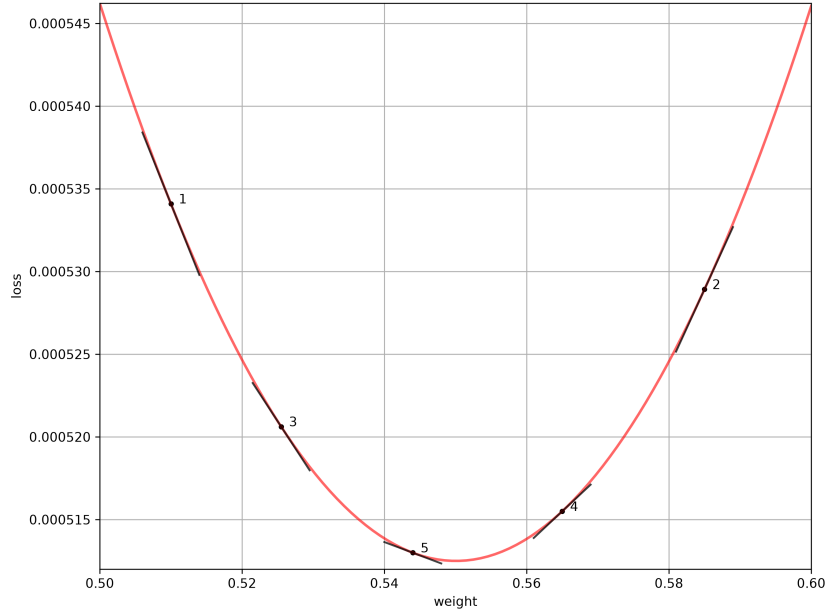


Figure 2: Gradient Descent with Fixed Bias

On the graph, the bias is fixed so that it is easier to visualize. Now, to minimize the loss, gradient descent is used. It works by finding the gradients of weight(s) and bias(es) (here it is fixed) and then moving in the direction of the steepest gradient, achieving the minimum of the function.

At the very start, the weight is random; it is around 0.51. The gradient is calculated and subtracted from the weight, because we want to minimize the loss. The same thing is repeated as many times as possible to achieve the smallest loss, and the gradient ensures that the loss is decreasing.

The gradient of weight in this graph is just the partial derivative of the loss by the weight. I will start of with writing down all of the equations we have:

$$L(y) = \frac{1}{2}(y - t)^2 \quad (1)$$

$$y = wx + b \quad (2)$$

Now we derive:

$$\partial L / \partial y = y - t \quad (3)$$

Partial derivative by w , so I treat b as a constant:

$$\partial y / \partial w = x$$

Then combine using chain rule:

$$\begin{aligned}\partial L / \partial w &= \partial L / \partial y \times \partial y / \partial w = \\ &= (y - t) \times x\end{aligned}\tag{4}$$

This is how the neural network knows how to change the weight. In order to minimize loss, we subtract the derivative from the weight.

$$w = w - \partial L / \partial w$$

However, the derivative might be a too big, and the weight could change too much. This could cause instability, as the weight could overshoot all the time, and it might never reach the minimum. To address this a learning rate constant is introduced - η . The derivative is multiplied by it too ensure that the steps are small so that the minimum is achieved no matter what. The formula for weight change becomes:

$$w = w - \partial L / \partial w \times \eta\tag{5}$$

η is usually somewhere around 0.1 to 0.001.

Now the same thing must be repeated with the bias, although I treated it as a constant for the sake of the clean visuals, in the process of learning the bias must also be adjusted.

The equations:

$$\begin{aligned}L &= \frac{1}{2}(y - t)^2 \\ y &= wx + b\end{aligned}$$

Derived previously:

$$\partial L / \partial y = y - t$$

Partial derivative by bias, treating weight as a constant.

$$\partial y / \partial b = 1$$

Chain rule:

$$\begin{aligned}\partial L / \partial b &= \partial L / \partial y \times \partial y / \partial b = \\ &= (y - t) \times 1 = \\ &= y - t\end{aligned}\tag{6}$$

And now the bias is changed:

$$b = b - \partial L / \partial b \times \eta \quad (7)$$

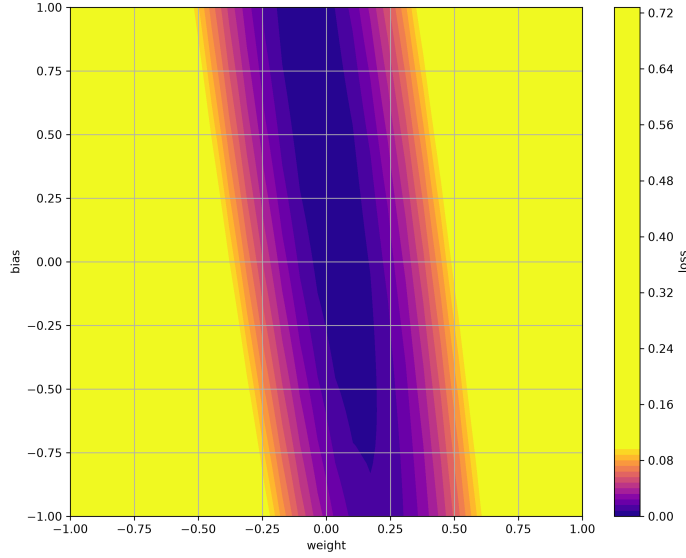


Figure 3: Countour Map of $L(w, b)$

This diagram illustrates the loss function with parameters of weight and bias, instead of fixing one of them. The x and y axis are weight and bias, respectively, and the color represents loss as shown on the right side with the colorbar.

$$L(w, b) = \frac{1}{2}((wx + b) - t)^2$$

Apart from showing the dependency of loss on weight and bias, this diagram also highlights the limitation of the learning method. Gradient decent finds the local minimum of loss function, however this particular local minimum that the model would converge to might not be the global minimum.

One of the things I have not mentioned yet is where I got the x and t from. As I said x is the living area, and t is the price of an apartment. I have collected data from funda.nl and I only picked the apartments that are in Amsterdam and that are for sale in July of 2026. In total, I have collected around 4500 apartments with their respective prices and features.

Because the function does calculation of only one apartment at a time, I took average of all losses for each apartment with a certain weight and bias and that is what is being presented on the previous graphs. That is why I have not

used x nor t as arguments for functions y and L , for the process of learning it is treated as a constant. The weight and bias are the independent variables that must be changed. After the network has finished its learning and, therefore, has constant weight and bias, the function will become $y(x) = wx + b$ and x will be the independent variable again.

2.3 Not Normalized

Another thing that must be done for a network to work is normalization of inputs and outputs. Here is an example, that will explain the importance of normalization:

I will take learning rate as 0.001, it is 10 times less than I normally would use because this will illustrate the influence of normalization clearer.

An apartment that is located at 1061 AV Amsterdam has price is 930 000 € and it has living area of 108 m^2 . Let w be 8700 and b be 348, those are values that achieve a decent amount of accuracy for this example. The function is:

$$y(x) = 8700 \times x + 348$$

x is the living area, so it is 108 m^2 . y is the predicted price:

$$y(x) = 8700 \times 108 + 348 = 939948 \text{ €}$$

t is the target price, so it is 930 000 €. Using t and y we can now calculate the loss:

$$\begin{aligned} L(x) &= \frac{1}{2}(y(x) - 930000)^2 \\ \Rightarrow L(108) &= \frac{1}{2}(939948 - 930000)^2 = \\ &= 4.95 \times 10^7 \end{aligned}$$

Now we calculate the gradient decent:

$$\begin{aligned} \partial L / \partial w &= (y - t) \times x = (939948 - 930000) \times 108 = 1074.4 \times 10^3 \\ w &= w - \partial L / \partial w \times \eta = (8700 - 1074.4 \times 10^3) \times 0.001 = 7625.616 \\ \partial L / \partial b &= (y - t) = (939948 - 930000) = 9.9 \times 10^3 \\ b &= b - \partial L / \partial b \times \eta = (348 - 9.9 \times 10^3) \times 0.001 = 338.052 \end{aligned}$$

And calculate the loss again with new weight and bias:

$$\begin{aligned} y(108) &= 7625.616 \times 108 + 338.052 = \\ &= 823904.58 \text{ €} \\ L(108) &= \frac{1}{2}(y - t)^2 = \frac{1}{2}(823904.58 - 930000)^2 = \\ &= 562.8 \times 10^7 \end{aligned}$$

The new loss is significantly larger than the first one: $4.95 \times 10^7 < 562.8 \times 10^7$. Although the gradient descent should have decreased the loss, it overshoot instead. The initial weights and biases were definitely wrong, as the first prediction was around 54 €. However, because of the huge inputs - 108 and the target outputs - 930 000, the gradients skyrocketed: 1074.4×10^3 , 9.9×10^3 . Although the learning rate was small - 0.001, the weight and bias still overshoot. This experiment can be repeated with any other weights and most of the time, the derivative will overshoot and the network will not minimize the loss function and therefore will not learn.

2.4 Normalization

To address this normalization of data is introduced. The goal is to fit any inputs and outputs into range from 0 to 1. For that this formula can be used:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (8)$$

This is done for every piece of data: both price and living area and other features. Normalization will ensure that weights are also in a small range, usually from -1 to 1, rarely might reach -2, or 2. This also allows for a larger learning rate, and 0.1 will be used here, otherwise the result will be difficult to see.

For my data the min and max for prices and living areas are respectively:

$$\begin{aligned} &189k - 9850k \text{ €} \\ &19 - 500 \text{ m}^2 \end{aligned}$$

Also, I divide the price by 10^3 to make the difference between prices more significant, and it is clearer to visualize the prices in thousand euros instead of just euros, therefore I will be using k € from now on.

I will repeat the same example but with normalized values now, the weight and biases are also chosen so that the function is somewhat accurate:

$$\begin{aligned} w &= 0.5 \\ b &= -0.03 \end{aligned}$$

$$\begin{aligned} x &= 108 \text{ m}^2 \\ t &= 930k \text{ €} \end{aligned}$$

$$\begin{aligned} x_{\text{norm}} &= (x - x_{\min}) / (x_{\max} - x_{\min}) \\ &= (108 - 19) / (500 - 19) = 0.185 \\ t_{\text{norm}} &= (t - t_{\min}) / (t_{\max} - t_{\min}) \\ &= (930 - 189) / (9850 - 189) = 0.0767 \end{aligned}$$

Since all of the values will be normalized from now on, I will not use the `_norm` suffix.

$$y = 0.5 \times x - 0.03 = 0.5 \times 0.185 - 0.03 = 0.0625$$

$$L = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.0625 - 0.0767)^2 = 1.006 \times 10^{-4}$$

$$\partial L / \partial w = (y - t) \times x = (0.0625 - 0.0767) \times 0.185 = -2.62 \times 10^{-3}$$

$$w = w - \partial L / \partial w \times \eta = 0.5 - -0.00262 \times 0.1 = 0.50026$$

$$\partial L / \partial b = (y - t) = 0.0625 - 0.0767 = -1.42 \times 10^{-2}$$

$$b = b - \partial L / \partial b \times \eta = -0.03 - -0.0142 \times 0.1 = -0.0286$$

$$y = wx + b = 0.50026 \times 0.185 - 0.0286 = 0.064$$

$$L = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.064 - 0.0767)^2 = 0.809 \times 10^{-4}$$

As can be seen here the loss decreased from 1.00×10^{-4} to 0.81×10^{-4} . Normalization allows for a more stable learning process and less overshooting during the gradient descent. Moreover, normalization helps with weights and biases initialization, as we can assume that they are going to be most frequently around -1.0 to 1.0. On the other hand, without normalization the weights might go to thousands like in the last example. The diagram below shows how centered the blue graph (normalized data) to the zero compared to the red graph (not normalized data). Furthermore, the gradient is much less steep for the blue graph, which again shows that the gradient descent will be more smooth and less overshooting.

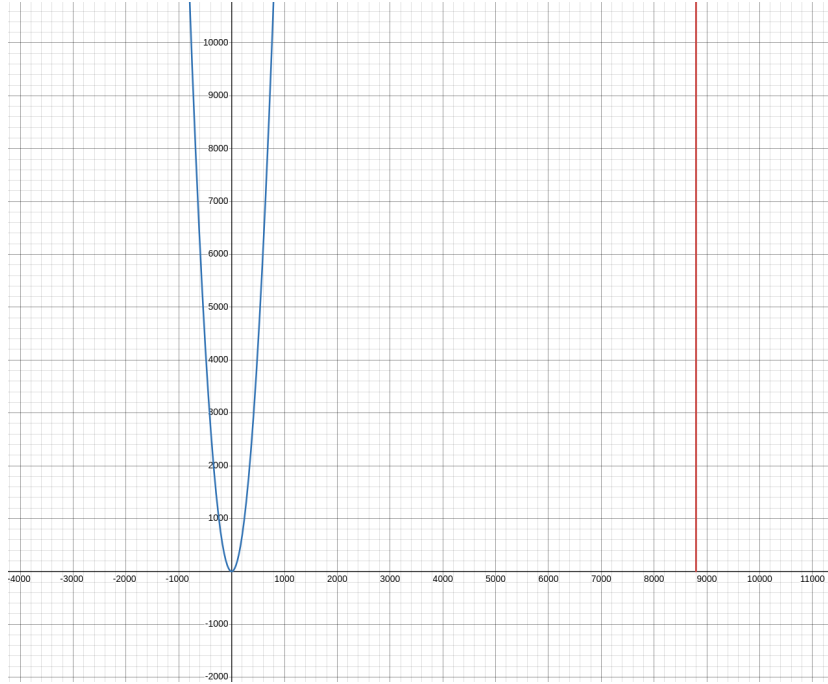


Figure 4: Normalized vs not Normalized

2.5 Learning Process

I have a few thousand apartments data from which my neural network must learn. The term epoch is introduced at this point, which refers to a single, complete pass of the entire training dataset through the model. In this case, gradient descent will be done for each apartment in the dataset. x is the living area, and t is the respective price for the apartments. This will make the neural network find the middle ground for weight and bias to minimize the loss across all the apartments. For a model to minimize loss as much as possible, numerous epochs are done. Depending on the length of the dataset this number can range, but I will be doing around 100-1000 epochs.

As I mentioned earlier, the dataset includes around 4500 entries; it is exactly 4577 apartments. I have separated the data into training and test datasets. The training entries are used to train the model, in the process mentioned above. On the other hand, the test dataset is used to test the accuracy of predictions of the model. The test process illustrates how good the network performs on apartments it has never seen before. To separate the dataset into training and test datasets, I will be using a 0.95 ratio - 4348 and 229 entries, respectively.

Here is the model I have trained on my dataset with the structure $y = wx + b$, learning rate 0.01 and 1000 epochs. Blue points are the entries from the dataset, and the red dots are the predictions made by the model.

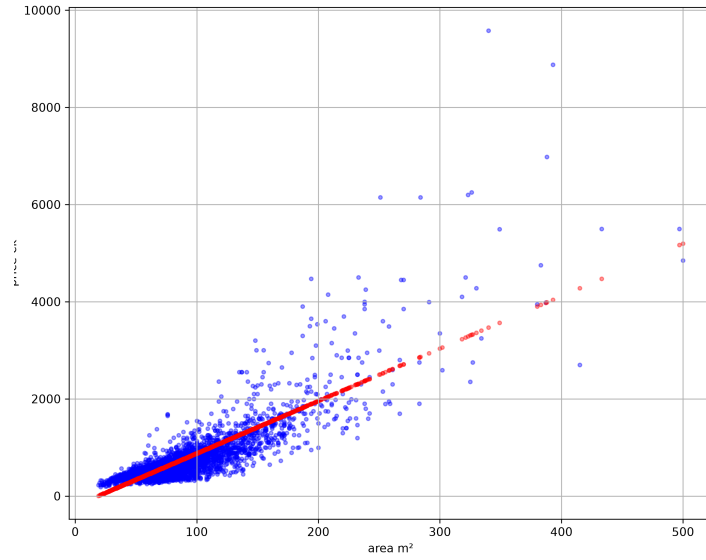


Figure 5: Model Results

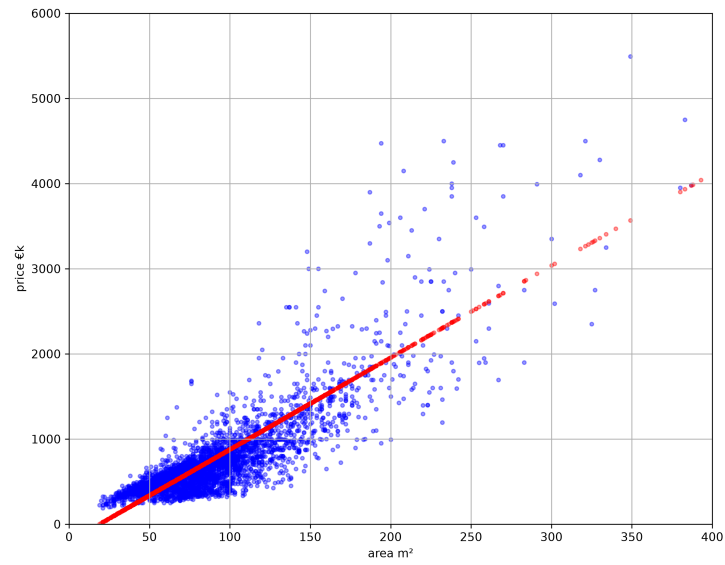


Figure 6: Model Results Without Outliers

This model has converged its weight and bias values to around 0.553 and -0.0197, respectively. This achieved the loss of 4.798×10^{-4} . I want to remind

you that in previous example I achieved less loss, 0.81×10^{-4} because the loss was calculated across one entry which practically has a certain weight and bias that achieves 0 loss. $y = wx + b$ where y and x are constants means that there are infinitely many solutions.

On the other hand, here are 4500 entries that are impossible to cover with a linear equation like this. This line that my model has learned is just the best perfect line. Furthermore, the blue dots are arranged in a mostly chaotic way with a tendency of the price increasing as the area increases. This suggests that only area is not enough to fully predict price of house.

Nevertheless, this model can predict housing, although accuracy is limited. I can build a function according to the weight and bias I got:

$$y(x) = 0.5497 \times x - 0.0186$$

Let's test it on this example:

$$\begin{aligned} \text{Area} &= 79 \text{ m}^2 \\ \text{Target price} &= 700\text{k €} \end{aligned}$$

First normalize the data:

$$x = \frac{79 - 19}{500 - 19} = 0.1247$$

Use the function:

$$\begin{aligned} y(x) &= 0.5497 \times x - 0.0186 \\ \Rightarrow y(0.1247) &= 0.5497 \times 0.1247 - 0.0186 = 0.0499 \end{aligned}$$

Now we can denormalize y by rearranging the normalization function:

$$\begin{aligned} y_{\text{norm}} &= \frac{y - y_{\min}}{y_{\max} - y_{\min}} \\ y_{\text{norm}} \times (y_{\max} - y_{\min}) &= y - y_{\min} \\ y &= y_{\text{norm}} \times (y_{\max} - y_{\min}) + y_{\min} \end{aligned} \tag{9}$$

$$y = 0.0493 \times (9850 - 189) + 189 = 671.46$$

The target was 700k and the model predicted 671.46k which is not too bad for a network that only has the information about living area.

To assess how good this model actually is, this process must be done for more than one apartment. I put this model through my test dataset, with the results I got I can calculate the mean absolute error MAE and mean percentage absolute error MAPE.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - t_i| \quad (10)$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - t_i|}{t_i} \quad (11)$$

I got $\text{MAE} = 162.0k$ and $\text{MAPE} = 25.5\%$. Here are few other examples of predictions this model has made:

Target t (k €)	359.0	450.0	475.0	650.0	685.0	880.0
Predicted y (k €)	454.1	357.5	378.9	604.3	840.4	1205.3

$3.749\text{e-}04$ is the least loss I was able to get while using only one feature of the apartments.

3 Two Input Neural Network

3.1 Training Process

The next step of minimizing loss is adding more information about the apartment for the network to use. I will be using the construction year as the second input.

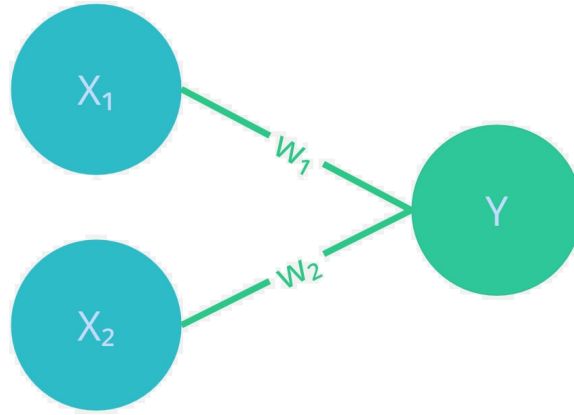


Figure 7: Neural Network with Two Inputs

As shown on the diagram, there are three neurons: two inputs and one output. Each input has its own weight with which it is connected to the output. The first input x_1 is the living area and the second input x_2 is the construction year of the apartments. Output y is the predicted price, that will not change. The bias is still present but there is no need for two biases (one for each input), only one is used. The function of y becomes:

$$y(x_1, x_2) = w_1x_1 + w_2x_2 + b$$

The loss $L(y)$ function (Equation 1) nor the $\partial L/\partial y$ (Equation 3) do not change. Weight derivatives will not change significantly and will be similar to (Equation 4). The partial derivative must be done for each weight, while every other weight and bias are treated as constants.

$$\partial L/\partial w_1 = (y - t) \times x_1$$

$$\partial L/\partial w_2 = (y - t) \times x_2$$

The partial derivative for bias stays the exactly same, Equation 6:

$$\partial L/\partial b = y - t$$

The second input will be the construction year of the apartment. Although it might not seem like an important feature of a house, the construction year suggests the quality of the materials and the electronics of the apartment. The data about that was also collected from funda.nl.

This diagram illustrates the dependency of price on the construction year of the apartment. There is a correlation that newer apartments are able to be more expensive than the old ones, however this tendency is much less clear than the one with area. This means that the area will have more impact on the price rather than the construction year.

This input like any other will also be normalized, the minimum year is 1507 and maximum is 2028. Most apartments in year range 2026-2028 weren't been built yet, however, I will still be using data about them to increase my dataset and thereby improve my model.

$$x_{2\text{norm}} = \frac{x_2 - 1507}{2028 - 1507}$$

I trained model with 1000 epochs, and it converged to weights and biases:

$$w_1 = 0.5502$$

$$w_2 = -0.0515$$

$$b = 0.2595$$

$$\begin{aligned} y(x_1, x_2) &= w_1x_1 + w_2x_1 + b \\ \Rightarrow y(x_1, x_2) &= 0.5502 * x_1 - 0.0515 * x_2 + 0.2595 \end{aligned}$$

Let's do the same example as I did with area:

$$\begin{aligned}\text{Area} &= 79 \text{ m}^2 \\ \text{Year} &= 2005 \\ \text{Target price} &= 700\text{k €}\end{aligned}$$

$$\begin{aligned}x_1 &= \frac{79 - 19}{500 - 19} = 0.1247 \\ x_2 &= \frac{2005 - 1507}{2028 - 1507} = 0.9559\end{aligned}$$

$$\begin{aligned}y(0.1247, 0.9559) &= \\ &= 0.5502 \times 0.1247 - \\ &\quad - 0.0515 \times 0.9559 + \\ &\quad + 0.2595 = \\ &= 0.0453\end{aligned}$$

De-normalize:

$$y = 0.0453 \times (9850 - 189) + 189 = 626.83\text{k €}$$

This model predicted 626k € with the target of 700k €. Although, on this entry this model performed worse than the last one, it has achieved a less average loss across the test dataset: 3.484e-04. This model achieved mean absolute error of 149.5k € and mean absolute percentage error 22.9%, which is a noticeable improvement compared to 162k € and 25.5% of the previous model. MAE and MAPE are calculated using formulas from Equations 10 and 11.

Here is the performance of this model on other examples:

Target t (k €)	359.0	450.0	475.0	650.0	685.0	880.0
Predicted y (k €)	428.1	408.6	403.1	624.1	896.6	1146.7

3.2 Influence Of Construction Year on the Results

According to the loss improvement, information about construction year does have a clear impact on the price of apartments. To measure significance of a feature we can look at the weights of the model. We only need the absolute values, as the negative sign just means that the correlation is inverse.

$$\begin{aligned}|w_1| &= 0.5502 \\ |w_2| &= 0.0515\end{aligned}$$

As I mentioned earlier, living area does have a bigger influence on price. Here is a diagram of this model results against living areas and construction years:

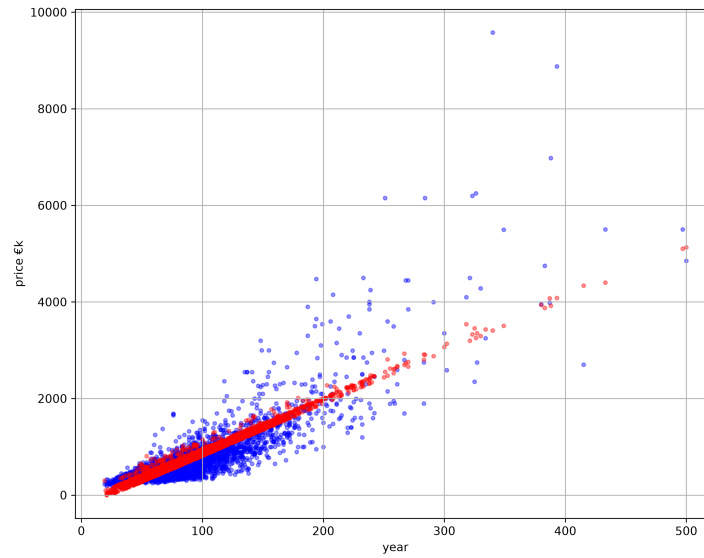


Figure 8: Area Against Price

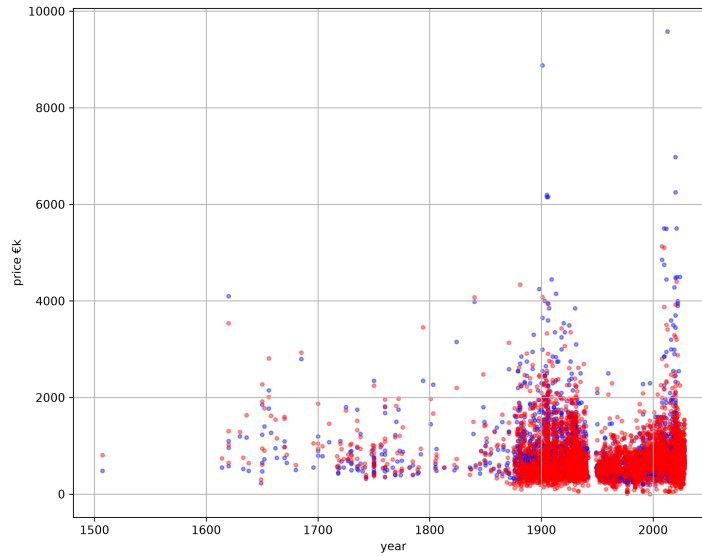


Figure 9: Year Against Price

As expected, model have built a best fit line for area just like last time. However, if we look at the construction year diagram, it does not look like a linear function, but rather a really chaotic spread that tries to resemble the actual price to year graph. The reason it looks that way is because having a function $y = w_1x_1 + w_2x_2 + b$ creates a plane in 3 dimensional space. Here is how this function looks in desmos 3d.

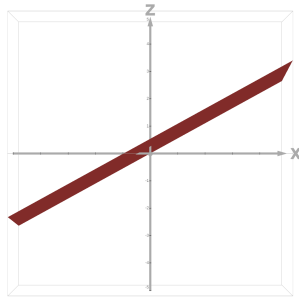


Figure 10: Area

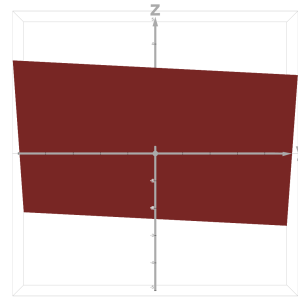


Figure 11: Year

From area price side it looks almost like one line, because $|w_2|$ is big, around

0.55. On the other hand, $|w_2|$ is small, around 0.05, and it creates a surface for construction year graph. This surface is what creates what looked like a random spread, the truth is that this surface looked random because of the dataset which has random data.

4 Neural Network with 3 Inputs

4.1 Calculate Distance to the Center of Amsterdam

Until this point, the information of the apartment has only contained the details of the building itself, however to further improve the model I decided to add information about the location of this apartment. As I mentioned earlier, all apartments in my dataset are located in Amsterdam, so I decided to use the distance between the apartment and the center of Amsterdam.

First of all, I chose a point which will be the center of Amsterdam. It has latitude of 52.3676 and longitude of 4.9041, and here is a google maps link.

In my dataset, I know the address of each apartment, that is what I am going to use to get coordinates of the apartments. In order to gather the coordinates of an apartment just from its address, I am using pdok, which is a website that provides geodatasets from the Dutch government. I have built a script so I used the pdok api but there is also a human friendly version that I will be using for this next example.

For this example, I will be using the apartment from previous illustrations, that is located at 1073 SB Amsterdam. I put the address into this website, and got latitude 52.3516 and longitude 4.8995.

Now to calculate the distance between those two points I will be using Haversine formula:

$$\begin{aligned}\phi &= \text{lat}_2 - \text{lat}_1 \\ \lambda &= \text{lon}_2 - \text{lon}_1 \\ a &= \sin^2\left(\frac{\phi}{2}\right) + \cos(\text{lat}_1) \times \cos(\text{lat}_2) \times \sin^2\left(\frac{\lambda}{2}\right) \\ c &= 2 \times \arcsin(\sqrt{a}) \\ d &= R \times c\end{aligned}$$

Independent values:

$$\begin{aligned}R &= 6371 \text{ km the radius of the Earth} \\ \text{lat}_1 &= 52.3516 \\ \text{lon}_1 &= 4.8995 \\ \text{lat}_2 &= 52.3676 \\ \text{lon}_2 &= 4.9041\end{aligned}$$

Calculate difference in latitude and longitude:

$$\begin{aligned}\phi &= \text{lat}_2 - \text{lat}_1 = 0.016 \\ \lambda &= \text{lon}_2 - \text{lon}_1 = 0.0046\end{aligned}$$

Use the Haversine formula:

$$\begin{aligned}a &= \sin^2\left(\frac{0.016}{2}\right) + \cos(52.3516) \times \cos(52.3676) \times \sin^2\left(\frac{0.0046}{2}\right) \\ &= 2.01 \times 10^{-8} \\ c &= 2 * \arcsin^2(\sqrt{(2.01 \times 10^{-8})}) = 2.8 \times 10^{-4} \\ d &= c * R = 1.806 \text{ km}\end{aligned}$$

We can check this result on omnicalculator which is a website that provides alongside its other functionalities, a distance calculator between two points on Earth. The result I got on the website was identical to what I have got, therefore the formula is correct.

4.2 Building the Neural Network with 3 Inputs

Now, I can calculate the distance to the center of Amstvedam for each apartment. On my dataset, I have got the minimum distance 0.07km and the maximum 9.98km. So to normalize the distance I will use:

$$x_{\text{norm}} = \frac{x - 0.07}{9.98 - 0.07}$$

Using the data about the distance from the center of Amsterdam, I can add a new input neuron for the neural network - x_3 . So, the function becomes:

$$y(x_1, x_2, x_3) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

From here I can get all the derivatives necessary, using derivatives from Equations 3, 4 and 6. As always, while deriviving partially, I treat other variables as constants:

$$\begin{aligned}\partial L / \partial w_1 &= (y - t) * x_1 \\ \partial L / \partial w_2 &= (y - t) * x_2 \\ \partial L / \partial w_3 &= (y - t) * x_3 \\ \partial L / \partial b &= y - t\end{aligned}$$

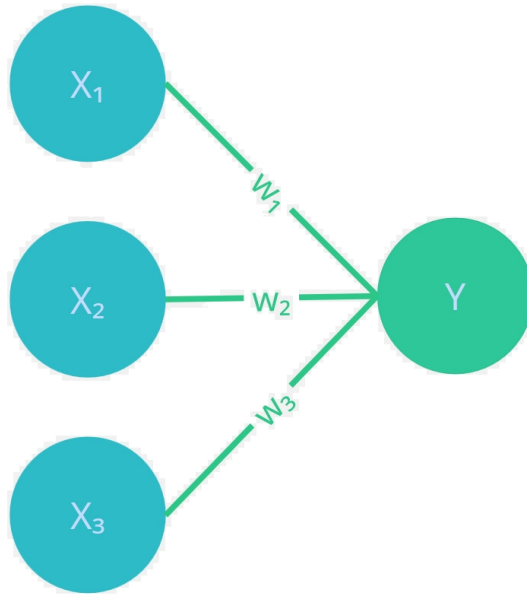


Figure 12: Neural Network with 3 Inputs

With this I can train a model for 1000 epochs. These are the weights and the bias, the neural network has converged to:

$$\begin{aligned}w_1 &= 0.5429 \\w_2 &= -0.0077 \\w_3 &= -0.0534 \\b &= 0.0072\end{aligned}$$

Let's test it on the previous example:

Area — 79 m^2
Year — 2005
Distance — 1.806 km
Target price — 700k €

Normalize inputs:

$$\begin{aligned}
x_1 &= \frac{79 - 19}{500 - 19} = 0.1247 \\
x_2 &= \frac{2005 - 1507}{2028 - 1507} = 0.9559 \\
x_3 &= \frac{1.806 - 0.07}{9.98 - 0.07} = 0.1752
\end{aligned}$$

Calculate:

$$\begin{aligned}
y(0.1247, 0.9559, 0.1752) &= 0.5429 \times 0.1247 + \\
&+ -0.0077 \times 0.9559 + \\
&+ -0.0534 \times 0.1752 + \\
&+ 0.0072 = \\
&= 0.0582
\end{aligned}$$

Denormalize according to Equation 9:

$$y \times (9850 - 189) + 189 = 751.34$$

This time the model overshoot the price, but this neural network has achieved the best results with the test dataset on average making 3.01×10^{-4} loss, and having the MAE and MAPE of 130.2 and 18.3%, respectively.

Here are some other results:

Target t (k €)	359.0	450.0	475.0	650.0	685.0	880.0
Predicted y (k €)	317.0	467.3	354.6	587.3	890.9	1105.1

4.3 Influence of Distance on Price

Just like last time we can measure the influence of each input by the absolute values of their weights:

$$\begin{aligned}
|w_1| &= 0.5429 \\
|w_2| &= 0.0077 \\
|w_3| &= 0.0534
\end{aligned}$$

We can see that the most influential input is still area, but the importance of construction year dramatically fell from 0.05 to 0.008, and the influence of distance is in middle of two other inputs.

Here is how the distribution of distance across the dataset looks like, (I have removed few outliers again):

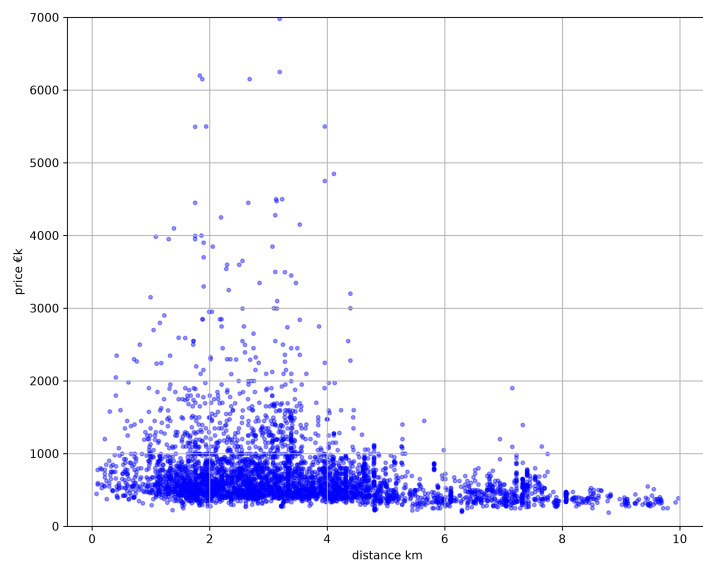


Figure 13: Distance against Price Data

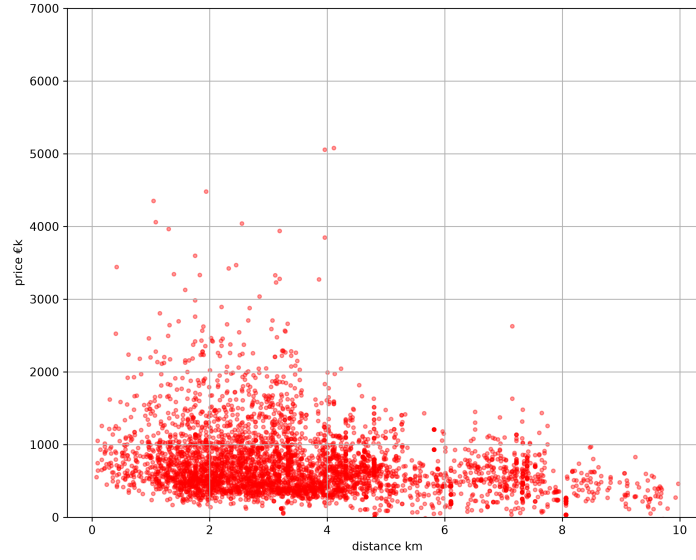


Figure 14: Distance against Price Predictions

The model was able to even capture the two spikes in price at distance 0 - 4km and 6 - 8km.

# of inputs	average loss	MAE	MAPE
1	3.7487×10^{-4}	162.0	25.5%
2	3.4844×10^{-4}	149.5	22.9%
3	3.0009×10^{-4}	130.2	18.3%